



영화 관객 수 예측 자료 시각화

미니 프로젝트(1)

YB 1조 김재경, 유재은, 이우림, 전예원

목차

1. 기본 설정

- 1) Google Drive 연결(마운팅)
- 2) 작업 디렉토리 변경 및 확인
- 3) 데이터 분석 및 시각화 라이브러리 불러오기
& 한글 깨짐 방지 설정
- 4) 경고 메시지 숨기기

2. 탐색적 자료분석(EDA)

- 1) 일부 데이터 미리보기
- 2) 데이터 요약
- 3) 데이터의 요약통계량 확인
- 4) 결측치 확인 및 데이터 전처리

3. 데이터 시각화 및 분석

- 1) 해당 감독이 이 영화를 만들기 전 제작에 참여한 영화에서의
평균 관객 수에 따른 관객 수
- 2) 개봉 월 시즌성 X 장르
- 3) 해당 감독이 이 영화를 만들기 전 제작에 참여한 영화의 수에
따른 관객 수
- 4) 개봉 연도별 전체 관객 수
- 5) 장르별로 개봉 연도에 따른 관객 수
- 6) 장르별 평균 관객수
- 7) 배급사 별 평균 관객 수
- 8) 영화 길이와 관객수의 관계
- 9) 스태프 수에 따른 관객 수
- 10) 배급사와 장르의 관계

1-1. Google Drive 연결(마운팅)

Google Colab에서 Google Drive를
사용할 수 있게 해주는 모듈을 불러옴

```
from google.colab import drive  
drive.mount('/content/drive/', force_remount=True)
```

이미 연결되어 있어도 강제로 다시 연결을 시도
하여 연결 오류를 줄이는 역할

1-2. 작업 디렉토리 변경 및 확인

* 연결된 Google Drive 내의 특정 폴더로 이동하여 작업을 준비하는 과정

파일 시스템을 조작할 수 있게 해주는 운영체제(OS) 관련 모듈을 불러옴

```
import os
os.chdir('/content/drive/MyDrive/ESAA/mini_project1/data')
os.getcwd()
```

현재 작업 디렉토리를 변경

1-3. 데이터 분석 및 시각화 라이브러리 불러오기 및 한글 깨짐 방지 설정

* 데이터 분석 및 시각화 라이브러리를 import 명령어로 불러옴

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
plt.rc('font', family='NanumBarunGothic') #matplotlib 한글 깨짐 방지
```

as로 라이브러리의
별명을 붙여줌

* Pandas: 표(테이블) 형태의 데이터를 다룸

* Numpy: 다차원 배열과 행렬 연산을 다룸

* Seaborn: 고급 데이터 시각화 라이브러리로 Matplotlib을 기반으로 하지만, 더 예쁘고 통계적인 그래프를 만들어줌

* Matplotlib: 가장 기본적인 데이터 시각화 라이브러리

1-4. 경고 메시지 숨기기

* 경고 메시지를 화면에 출력하지 않도록 숨겨서 코드를 실행하는 환경을 깔끔하게 유지함

```
import warnings
warnings.filterwarnings(action='ignore')
```

모든 경고 메시지에 대해
취할 행동을 무시로 지정

2-1. 일부 데이터 미리보기

```
train=pd.read_csv('movies_train.csv')
```

```
train.head()
```

	title	distributor	genre	release_time	time	screening_rat	director	dir_prev_bfnum	dir_prev_num	num_staff	num_actor	box_off
0	개들의 전쟁	롯데엔터테인먼트	액션	2012-11-22	96	청소년 관람 불가	조병옥	NaN	0	91	2	1
1	내부자들	(주)쇼박스	느와르	2015-11-19	130	청소년 관람 불가	우민호	1161602.5	2	387	3	70
2	은밀하게 위대하게	(주)쇼박스	액션	2013-06-05	123	15세 관람가	장철수	220775.2	4	343	4	69
3	나는공부원이다	(주)NEW	코미디	2012-07-12	101	전체 관람가	구자홍	23894.0	2	20	6	2

* title : 영화의 제목

* distributor : 배급사

* genre : 장르

* release_time : 개봉일

* time : 상영시간(분)

* screening_rat : 상영등급

* director : 감독이름

* dir_prev_bfnum : 해당 감독이 이 영화를 만들기 전 제작에
참여한 영화에서의 평균 관객수
(단 관객수가 알려지지 않은 영화 제외)

* dir_prev_num : 해당 감독이 이 영화를 만들기 전 제작에
참여한 영화의 개수
(단 관객수가 알려지지 않은 영화 제외)

* num_staff : 스텝수

* num_actor : 주연배우수

* box_off_num : 관객수

* director : 감독이름

2-2. 데이터 요약

- * 600개의 데이터로 학습 시키고
243개의 영화의 관객수를 예측해야 함

```
print(train.shape)|
print(test.shape)
print(submission.shape)

(600, 12)
(243, 11)
(243, 2)
```

- * train.info(): 각 컬럼의 Null값의 개수와 데이터타입 파악

```
train.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 600 entries, 0 to 599
Data columns (total 12 columns):
#   Column                Non-Null Count  Dtype
---  -
0   title                 600 non-null   object
1   distributor           600 non-null   object
2   genre                 600 non-null   object
3   release_time         600 non-null   object
4   time                 600 non-null   int64
5   screening_rat        600 non-null   object
6   director             600 non-null   object
7   dir_prev_bfnum       270 non-null   float64
8   dir_prev_num         600 non-null   int64
9   num_staff            600 non-null   int64
10  num_actor            600 non-null   int64
11  box_off_num          600 non-null   int64
dtypes: float64(1), int64(5), object(6)
memory usage: 56.4+ KB
```


2-3. 데이터의 요약통계량 확인

```
train.describe()
```

Out[]:

	time	dir_prev_bfnum	dir_prev_num	num_staff	num_actor	box_off_num
count	600.0	270.0	600.0	600.0	600.0	600.0
mean	100.9	1050442.9	0.9	151.1	3.7	708181.8
std	18.1	1791408.3	1.2	165.7	2.4	1828005.9
min	45.0	1.0	0.0	0.0	0.0	1.0
25%	89.0	20380.0	0.0	17.0	2.0	1297.2
50%	100.0	478423.6	0.0	82.5	3.0	12591.0
75%	114.0	1286568.6	2.0	264.0	4.0	479886.8
max	180.0	17615314.0	5.0	869.0	25.0	14262766.0

* train.describe(): 평균, 표준편차, 최솟값, 최댓값, 제1사분위수, 제2사분위수, 제3사분위수를 보여줌

2-4. 결측치 확인 및 데이터 전처리

```
train.isna().sum()
```

Out[]:

title	0
distributor	0
genre	0
release_time	0
time	0
screening_rat	0
director	0
dir_prev_bfnum	330
dir_prev_num	0
num_staff	0
num_actor	0
box_off_num	0
dtype: int64	

* .isna(): 데이터프레임의 모든 값에 대해 NaN인지 검사함

-> 원본 데이터프레임과 동일한 크기의 True/False 값으로 채워진
불린 데이터프레임이 만들어짐 (결측치면 True, 아니면 False)

* .sum(): True는 숫자 1로, False는 숫자 0으로 간주하여 합계를 계산함

-> 각 컬럼에 있는 NaN의 개수를 알려줌

* train 데이터의 dir_prev_bfnum 변수에 결측값이 존재함

2-4. 결측치 확인 및 데이터 전처리

```
# 결측값의 특성 확인
train[train['dir_prev_bfnum'].isna()].head(3)
```

	title	distributor	genre	release_time	time	screening_rat	director	dir_prev_bfnum	dir_prev_num	num_staff	num_actor	box_o
0	개들의 전쟁	롯데엔터테인먼트	액션	2012-11-22	96	청소년 관람 불가	조병옥	nan	0	91	2	
6	길 위에서	백두대간	다큐멘터리	2013-05-23	104	전체 관람가	이창재	nan	0	32	5	
8	1789, 바스티유의 연인들	유니버설픽처스인터내셔널코리아	뮤지컬	2014-09-18	129	전체 관람가	정성복	nan	0	3	5	

* 전체 코드에 대한 합이 0이 나옴

-> 감독의 이전 흥행작 수가 결측치인 영화들의 감독의 총 이전작 수를 모두 더했더니, 그 총합이 0이 됨

```
print(train[train['dir_prev_bfnum'].isna()]['dir_prev_num'].sum())
```

0

=> 감독의 이전작품이 존재하지 않음

```
train['dir_prev_bfnum'].fillna(0, inplace=True)
```

* dir_prev_bfnum에 있는 Nan은 전부 0으로 채움

2-4. 결측치 확인 및 데이터 전처리

```
train.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 600 entries, 0 to 599
Data columns (total 12 columns):
#   Column                Non-Null Count  Dtype  
---  -
0   title                 600 non-null   object 
1   distributor           600 non-null   object 
2   genre                 600 non-null   object 
3   release_time          600 non-null   object 
4   time                  600 non-null   int64  
5   screening_rat         600 non-null   object 
6   director              600 non-null   object 
7   dir_prev_bfnum        270 non-null   float64 
8   dir_prev_num          600 non-null   int64  
9   num_staff             600 non-null   int64  
10  num_actor             600 non-null   int64  
11  box_off_num           600 non-null   int64  
dtypes: float64(1), int64(5), object(6)
memory usage: 56.4+ KB
```

```
train['release_time']=pd.to_datetime(train['release_time'])
```

* release_time을 object -> datetime형식으로 변환함

- release_time은 시간에 관련된 데이터인데 Dtype이 object로 나옴

3-1. 감독의 과거 성과와 현재 작품 흥행 간의 관계

1. 목적: 데이터 기반으로 감독 트랙레코드와 현재 성과 간의 상관관계 탐색

2. 데이터 및 전처리

- 데이터 변수

- `dir_prev_bfnum`: 감독의 과거 작품 총 관객 수
- `dir_prev_num`: 감독의 전작 편수
- `box_off_num`: 현재 작품 관객 수

- 로그 변환

- 극단적인 분포 완화 → `log1p()` 적용 (0도 안전하게 변환 가능)

- 전작 수 구간화

- 1편 / 2편 / 3편 / 4-5편 이상

```
# 로그 변환 (0 처리 안전)
```

```
train["log_prev"] = np.log1p(train["dir_prev_bfnum"])
```

```
train["log_aud"] = np.log1p(train["box_off_num"])
```

```
# 전작 수 구간화
```

```
bins = [1, 2, 3, 4, np.inf] # 1편, 2편, 3편, 4-5편
```

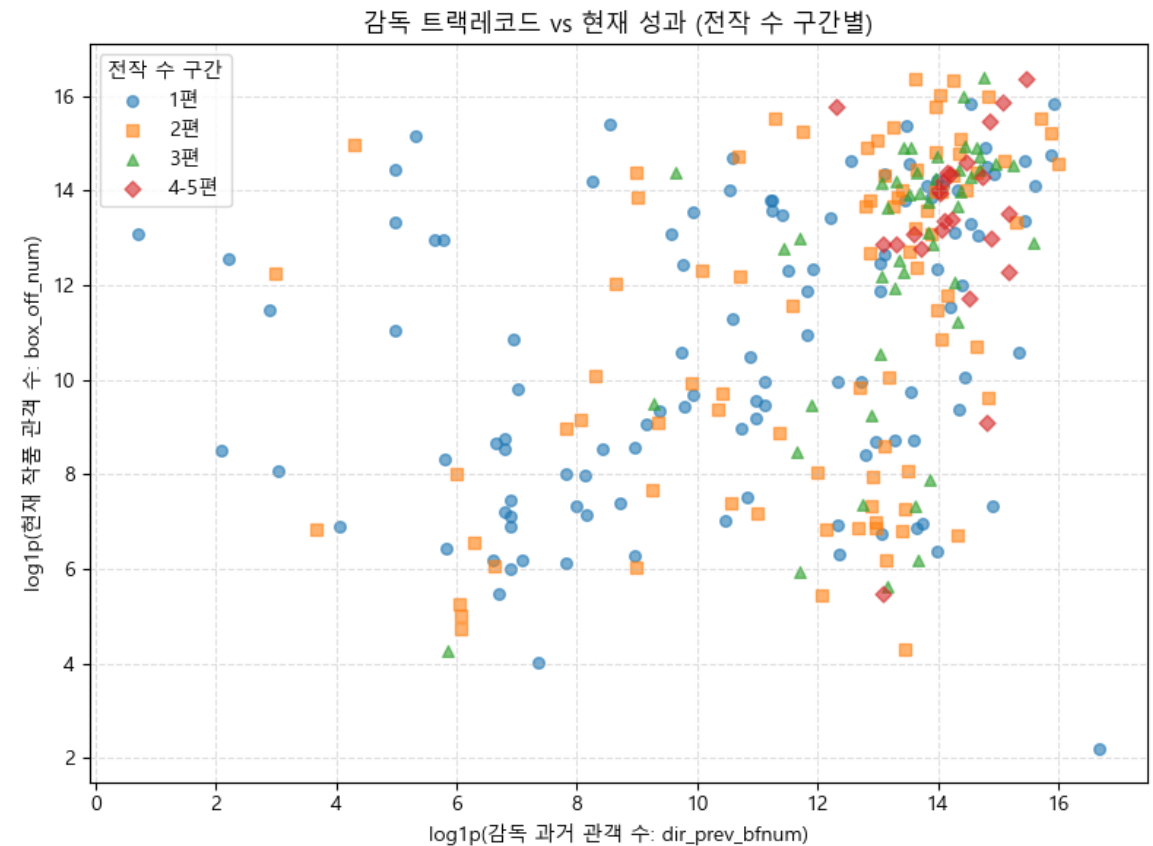
```
labels = ["1편", "2편", "3편", "4-5편"] # 라벨
```

```
train["prev_bins"] = pd.cut(train["dir_prev_num"], bins=bins, labels=labels, right=False)
```

3-1. 감독의 과거 성과와 현재 작품 흥행 간의 관계

3. 시각화(Scatter Plot)

- X축: $\log_{10}$ (감독 과거 관객 수)
- Y축: $\log_{10}$ (현재 작품 관객 수)
- 색/마커: 전작 수 구간 (1편, 2편, 3편, 4-5편)
- 그래프 해석 포인트
 - 전작 수 구간별로 분포가 어떻게 다른지
 - 특정 구간에서 두 변수의 관계가 강하게 드러나는지



3-1. 감독의 과거 성과와 현재 작품 흥행 간의 관계

4. 수치 요약

- 상관계수: log_prev vs log_aud $\rightarrow r = \{\text{corr:.3f}\}$
- 전작 수 구간 분포(빈도표)
 - 각 구간에 몇 명의 감독이 속하는지 보여줌

```
corr = train[["log_prev", "log_aud"]].corr().iloc[0,1]
print(f"상관계수 (log_prev vs log_aud): {corr:.3f}")
print(train["prev_bins"].value_counts().sort_index())
```

상관계수 (log_prev vs log_aud): 0.448

prev_bins

1편 113

2편 86

3편 47

4-5편 24

Name: count, dtype: int64

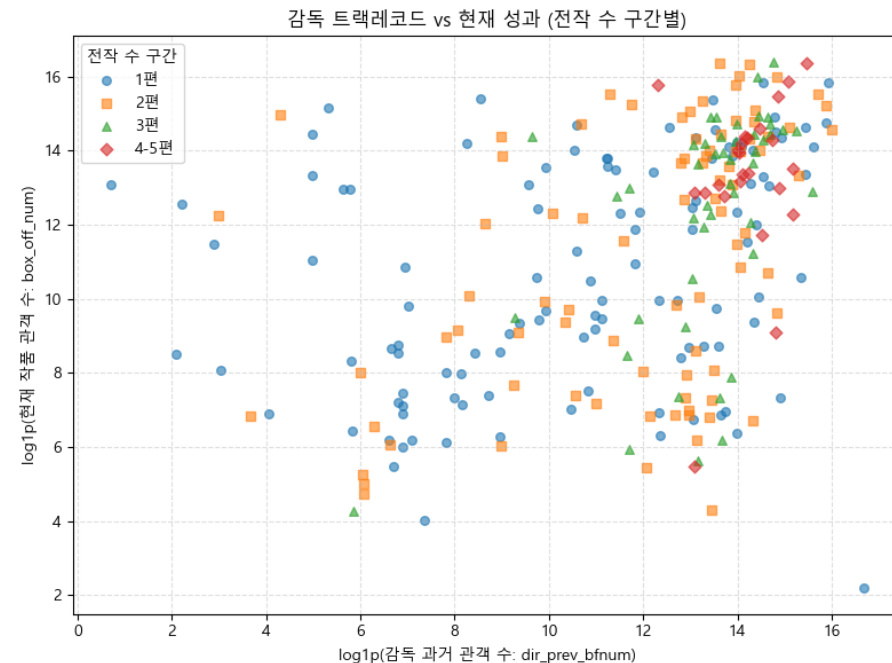
3-1. 감독의 과거 성과와 현재 작품 흥행 간의 관계

5. 해석

- 상관계수 해석: 양(+)의 상관 → 과거 성과가 좋을수록 현재 성과도 좋은 경향
- 전작 수 구간 해석:
 - 전작 1~2편 감독: 분산이 크고 불안정
 - 전작 3편 이상 감독: 안정적이며 일정 수준 이상의 성과 유지 가능성

6. 결론 및 시사점

- 결론: 감독의 과거 성과와 현재 성과 간에는 일정한 양의 관계가 존재
- 시사점:
 - 투자/기획 단계에서 감독의 전작 성과를 참고 지표로 활용 가능
 - 그러나, 영화 성공 요인은 다변적이므로 다른 변수 고려도 필요



3-2. 개봉 월 시즌성 X 장르

1. 목적: 영화 장르는 계절과 밀접한 연관이 있을까?

2. 데이터 전처리

- 개봉 월 추출: `release_time` → `month`
- 로그 변환: `log1p(관객수)` → 극단치 완화
- 상위 K개 장르만 선택 (K=6): 혼잡도 줄이고 대표 장르만 비교

```
# 파생: 개봉 월, 로그 관객수
train["month"] = train["release_time"].dt.month
train["log_aud"] = np.log1p(train["box_off_num"].astype(float))

# -----
# 1) 상위 K개 장르만 선택(혼잡도 줄이기)
# -----
TOP_K = 6
top_genres = train["genre"].value_counts().head(TOP_K).index
sub = train[train["genre"].isin(top_genres)].copy()
```

3-2. 개봉 월 시즌성 X 장르

3. 분석 방법

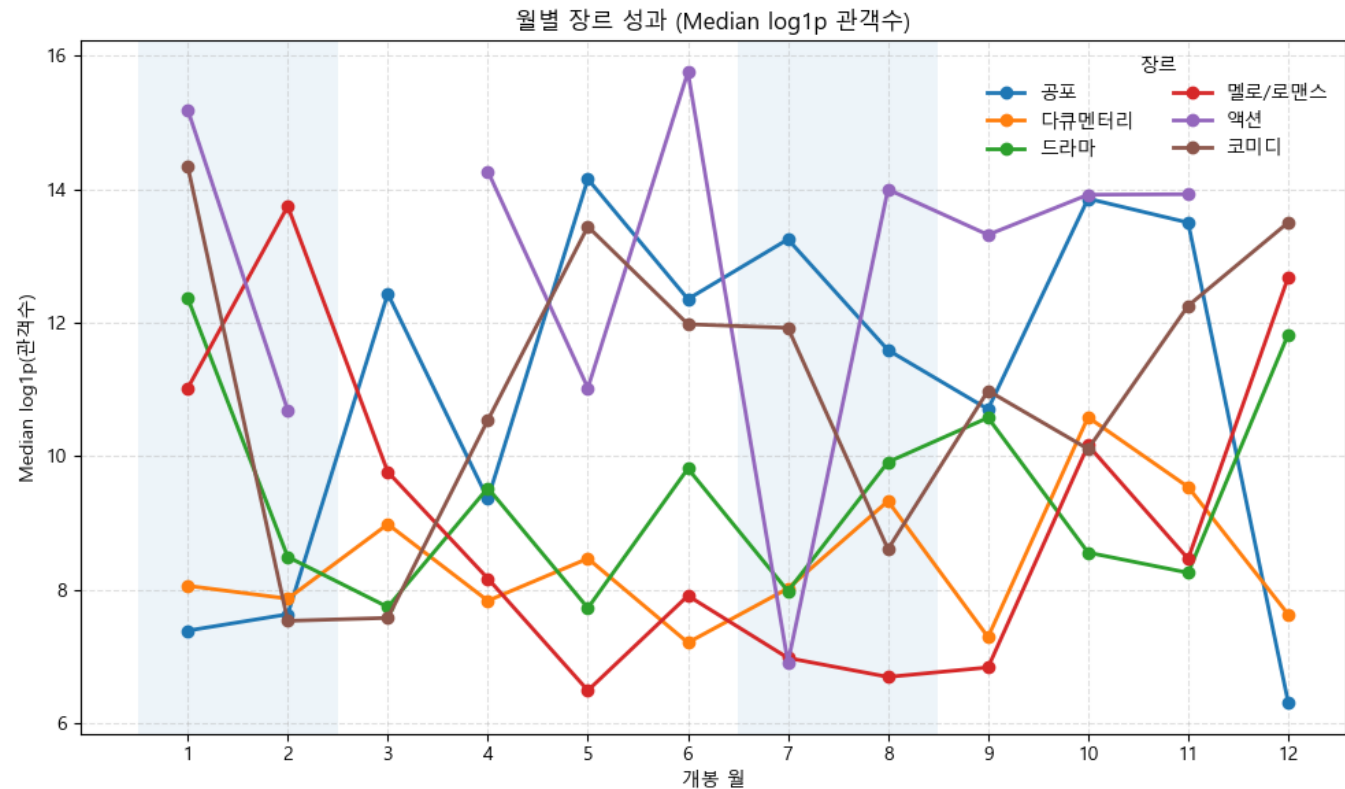
- 집계 기준:
 - 월별·장르별 중앙값 관객수 (median)
 - 표본 수(n)도 함께 계산
- 라인플롯:
 - x축: 개봉월(1-12)
 - y축: Median log1p(관객수)
 - 선: 장르별 추세
- 보조선: 방학/명절 구간(1-2월, 7-8월) 배경 밴드 표시

```
# -----  
# 2) 월 x 장르 집계(중앙값 + 표본수)  
# -----  
agg_median = (  
    sub.groupby(["genre", "month"])["log_aud"]  
        .median()  
        .rename("median_log_aud")  
        .reset_index()  
)  
agg_count = (  
    sub.groupby(["genre", "month"])["box_off_num"]  
        .size()  
        .rename("n")  
        .reset_index()  
)  
summary = pd.merge(agg_median, agg_count, on=["genre", "month"], how="left")  
  
# 표 확인  
print(summary.sort_values(["genre", "month"]).head(20))
```

3-2. 개봉 월 시즌성 X 장르

4. 시각화

- 주요 관찰 포인트:
 - 특정 장르가 방학·명절 시즌에 뚜렷하게 상승?
 - 장르별 계절 패턴 차이



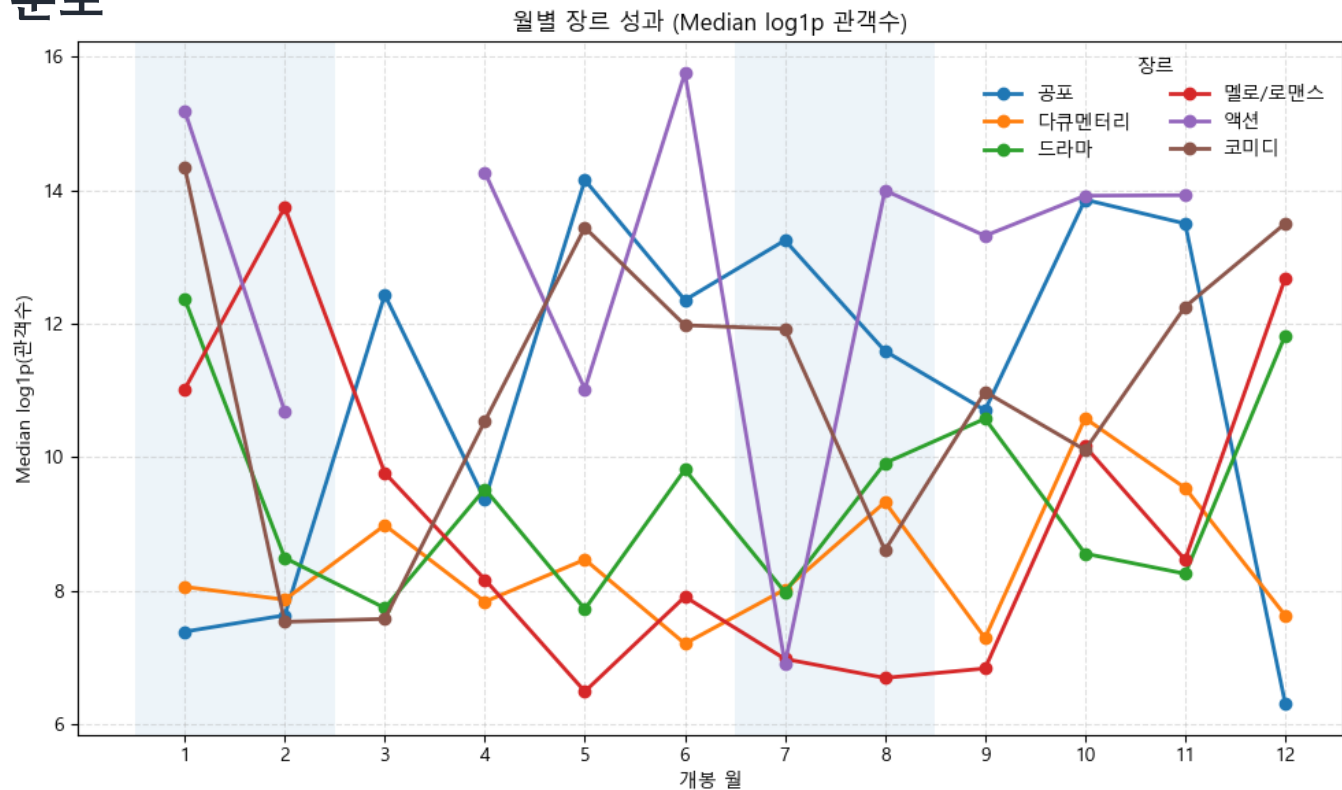
3-2. 개봉 월 시즌성 X 장르

5. 해석

- 방학/명절 시즌 -> 가족·애니메이션 장르 상승
- 여름 시즌 -> 액션·블록버스터 장르 강세
- 드라마/로맨스 -> 뚜렷한 계절성보다는 꾸준한 분포

6. 결론 및 시사점

- 결론: 장르별로 계절 효과가 존재
- 시사점:
 - 배급/마케팅 전략 수립에 활용 가능
 - 특정 시즌에 맞춘 장르 선택 -> 확률 ↑



3-3. 해당 감독이 이 영화를 만들기 전 제작에 참여한 영화의 수에 따른 관객 수

boxplot

사용 변수: dir_prev_num, box_off_num

x축: dir_prev_num

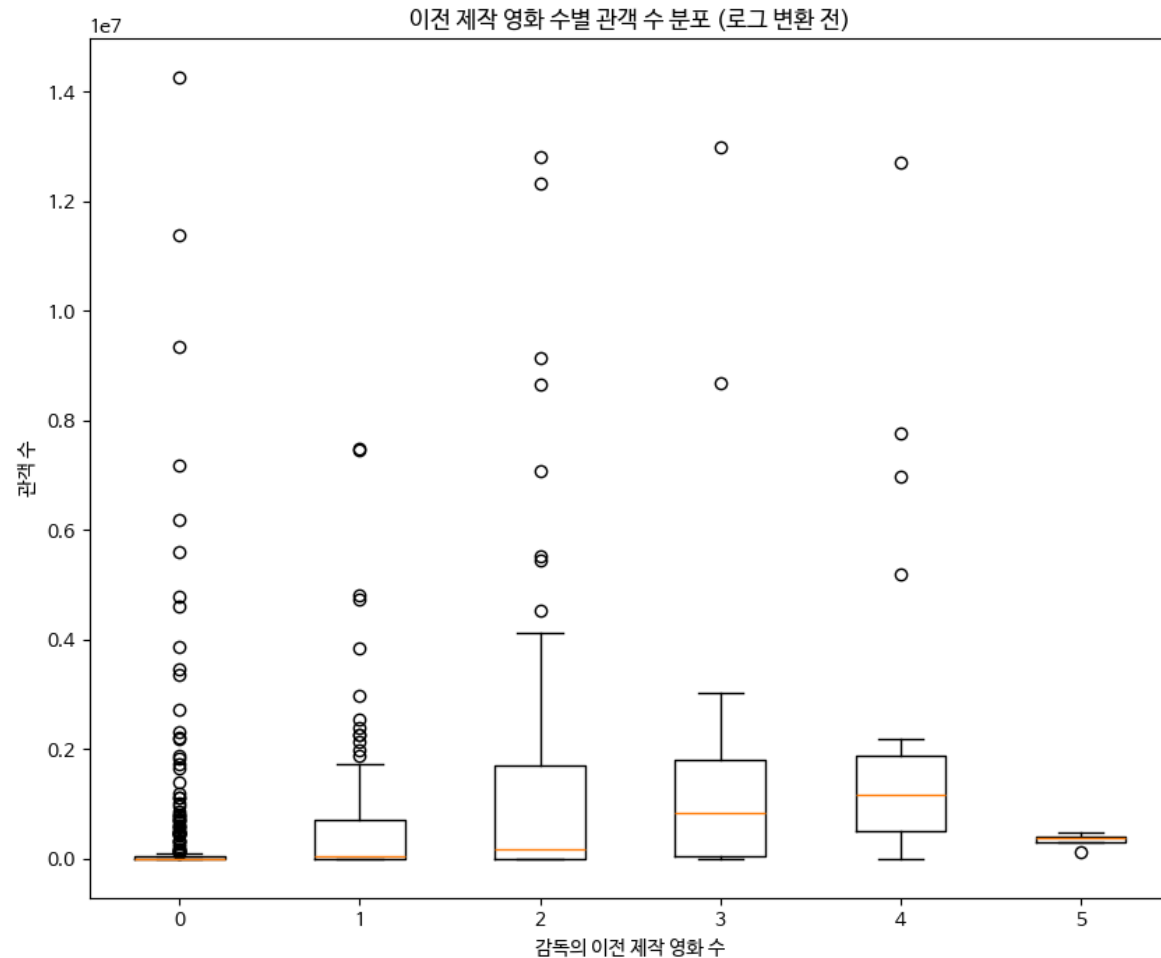
y축: box_off_num

```
plt.figure(figsize=(10,8))
plt.boxplot(
    [train.loc[train["dir_prev_num"] == i, "box_off_num"] for i in sorted(train["dir_prev_num"].unique())],
    labels = sorted(train["dir_prev_num"].unique())
)

plt.title("이전 제작 영화 수별 관객 수 분포 (로그 변환 전)", fontsize=12)
plt.xlabel("감독의 이전 제작 영화 수")
plt.ylabel("관객 수")
plt.show()
```

train['dir_prev_num']==i를 만족하는 행의 box_off_num 값 추출

3-3. 해당 감독이 이 영화를 만들기 전 제작에 참여한 영화의 수에 따른 관객 수



boxplot에서 이상치가 많고 데이터의 분포가 한쪽으로 치우쳐 있음
-> 극단값의 영향을 완화시키고 분포 안정화를 위해 box_off_num에 로그변환 수행

3-3. 해당 감독이 이 영화를 만들기 전 제작에 참여한 영화의 수에 따른 관객 수

boxplot

사용 변수: dir_prev_num, box_off_num

x축: dir_prev_num

y축: log_box_off_num (box_off_num 로그변환)

```
train["log_box_off_num"] = np.log1p(train["box_off_num"])
```

로그변환 수행 후 'log_box_off_num'에 저장

```
plt.figure(figsize=(10,8))
```

```
plt.boxplot(
```

```
    [train.loc[train["dir_prev_num"] == i, "log_box_off_num"] for i in sorted(train["dir_prev_num"].unique())],
```

```
    labels = sorted(train["dir_prev_num"].unique())
```

```
)
```

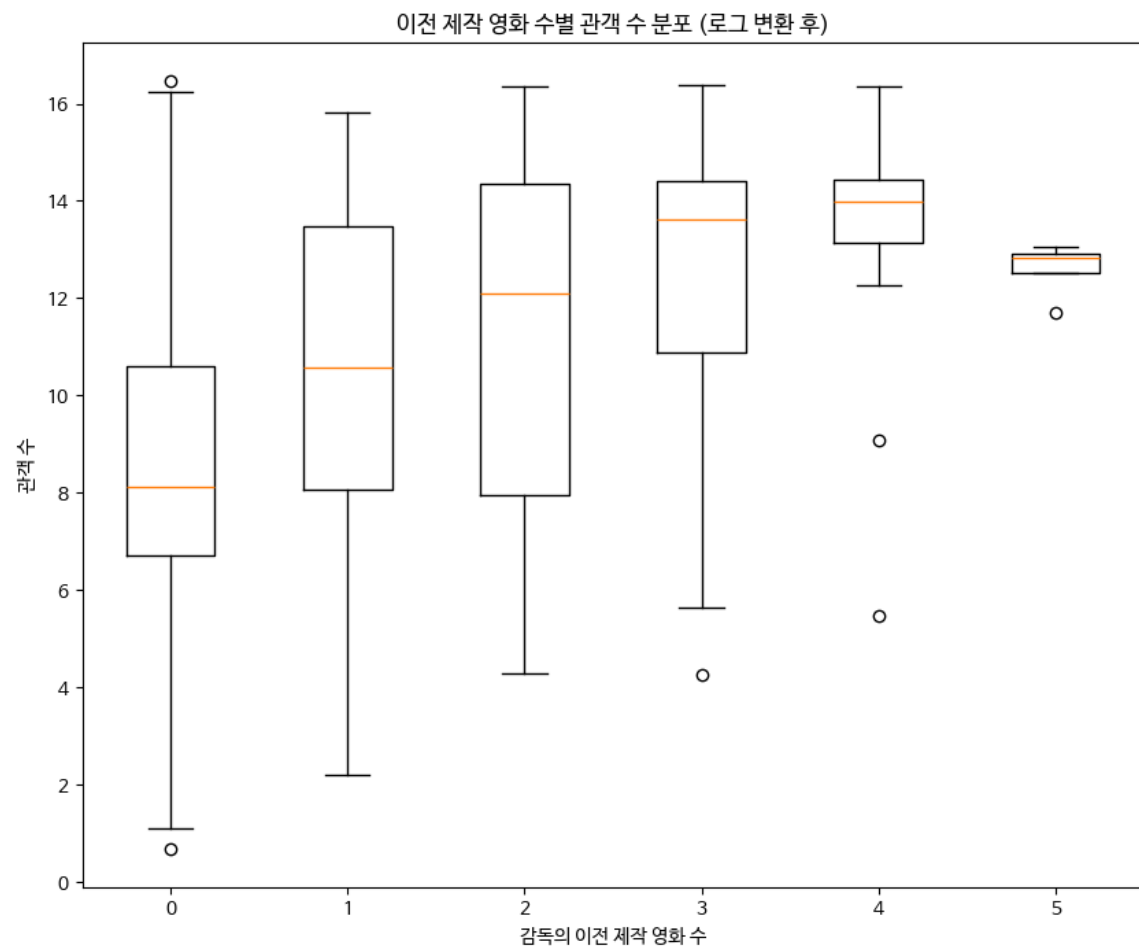
```
plt.title("이전 제작 영화 수별 관객 수 분포 (로그 변환 후)", fontsize=12)
```

```
plt.xlabel("감독의 이전 제작 영화 수")
```

```
plt.ylabel("관객 수")
```

```
plt.show()
```

3-3. 해당 감독이 이 영화를 만들기 전 제작에 참여한 영화의 수에 따른 관객 수



중앙값을 보면 감독의 이전 제작 영화 수가 많을수록 관객 수가 증가하는 경향이 있다.

-> 감독 경험이 많을수록 안정적인 흥행을 기대할 수 있음을 시사

그러나 boxplot의 수염과 이상치를 보면, 경험이 적은 감독도 일부 영화에서 높은 관객 수를 기록한 경우가 있다.

따라서 전반적으로 중앙값 기준으로는 증가 경향이 뚜렷하지만, 분포가 넓어 개별 사례의 변동성은 크다.

3-4. 개봉 연도별 전체 관객 수

barplot + lineplot

- 연도별 총 관객 수를 나타내는 barplot + 연도별 평균 관객 수를 나타내는 lineplot

barplot

사용 변수: release_time, box_off_num

x축: release_year

y축: yearly_sum

- release_time에서 year 추출하여 x축으로 지정
- release_year별로 관객 수의 합을 y축으로 지정
- 개봉 연도별 관객 수 합을 barplot으로 시각화

```
train['release_year'] = train['release_time'].dt.year
```

release_year에서 year 추출

```
yearly_sum = train.groupby("release_year")["box_off_num"].sum()
```

```
yearly_mean = train.groupby("release_year")["box_off_num"].mean()
```

```
fig, ax1 = plt.subplots(figsize=(10,8))
```

```
ax1.bar(yearly_sum.index, yearly_sum.values, color='skyblue', edgecolor='black')
```

```
ax1.set_xlabel("개봉 연도")
```

```
ax1.set_ylabel("총 관객 수")
```

```
ax1.tick_params(axis='y')
```

3-4. 개봉 연도별 전체 관객 수

lineplot

사용 변수: release_time, box_off_num

x축: release_year

y축: yearly_mean

- release_time에서 year 추출하여 x축으로 지정
- release_year별로 관객 수의 평균을 y축으로 지정
- 개봉 연도별 관객 수의 평균을 lineplot으로 시각화

총합과 평균의 스케일 차이를 고려해, twinx()를 사용하여 한 그래프에 두 y축을 겹쳐 표현

```
ax2 = ax1.twinx()
```

```
ax2.plot(yearly_mean.index, yearly_mean.values, color='red', marker='o', linewidth=2)
```

```
ax2.set_ylabel("평균 관객 수")
```

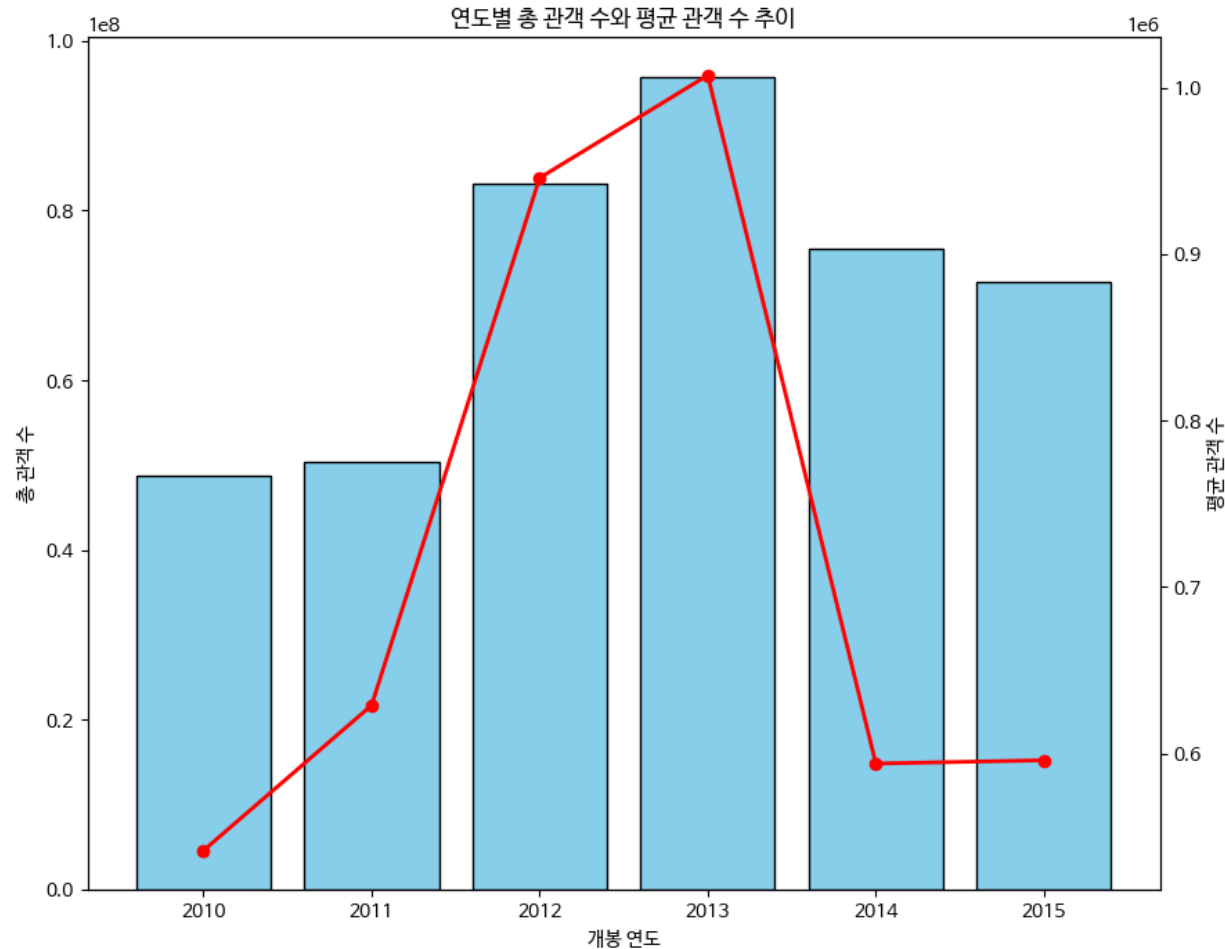
```
ax2.tick_params(axis='y')
```

```
plt.title("연도별 총 관객 수와 평균 관객 수 차이", fontsize=12)
```

```
plt.xticks(rotation=45)
```

```
plt.show()
```

3-4. 개봉 연도별 전체 관객 수



barplot을 보면 각 연도의 총 관객 수를 알 수 있다. 2013년에 총 관객 수가 가장 많았고, 다음은 2012, 2014 순으로 많았던 것을 확인할 수 있다.

lineplot을 보면 연도별 관객 수의 평균적인 수준을 보여준다. 2013년에 관객 수의 평균이 가장 높았고, 다음은 2012, 2011 순으로 높았던 것을 확인할 수 있다.

barplot과 lineplot을 함께 보면, 총합과 평균이 모두 높은 2012, 2013년에는 개봉 영화 수가 많고 흥행작도 많았을 것이라고 예측할 수 있다. 반면, 총합은 높으나 평균이 낮은 2014, 2015년의 경우에는 개봉 영화 수는 많으나 흥행력이 낮다는 것을 알 수 있다.

3-5. 장르별로 개봉 연도에 따른 관객 수

lineplot

사용 변수: release_time, genre, box_off_num

x축: release_year

y축: log_box_off_num

장르와 연도별로 그룹화

```
genre_yearly = train.groupby(['genre', 'release_year'])['log_box_off_num'].mean().reset_index()
```

```
pivot_df = genre_yearly.pivot(index='release_year', columns='genre', values='log_box_off_num')
```

```
plt.figure(figsize=(10,8))
```

```
ax = plt.gca()
```

```
pivot_df.plot(ax=ax, marker='o')
```

```
ax.set_title("장르별 연도별 평균 관객 수", fontsize=12)
```

```
ax.set_xlabel("개봉 연도")
```

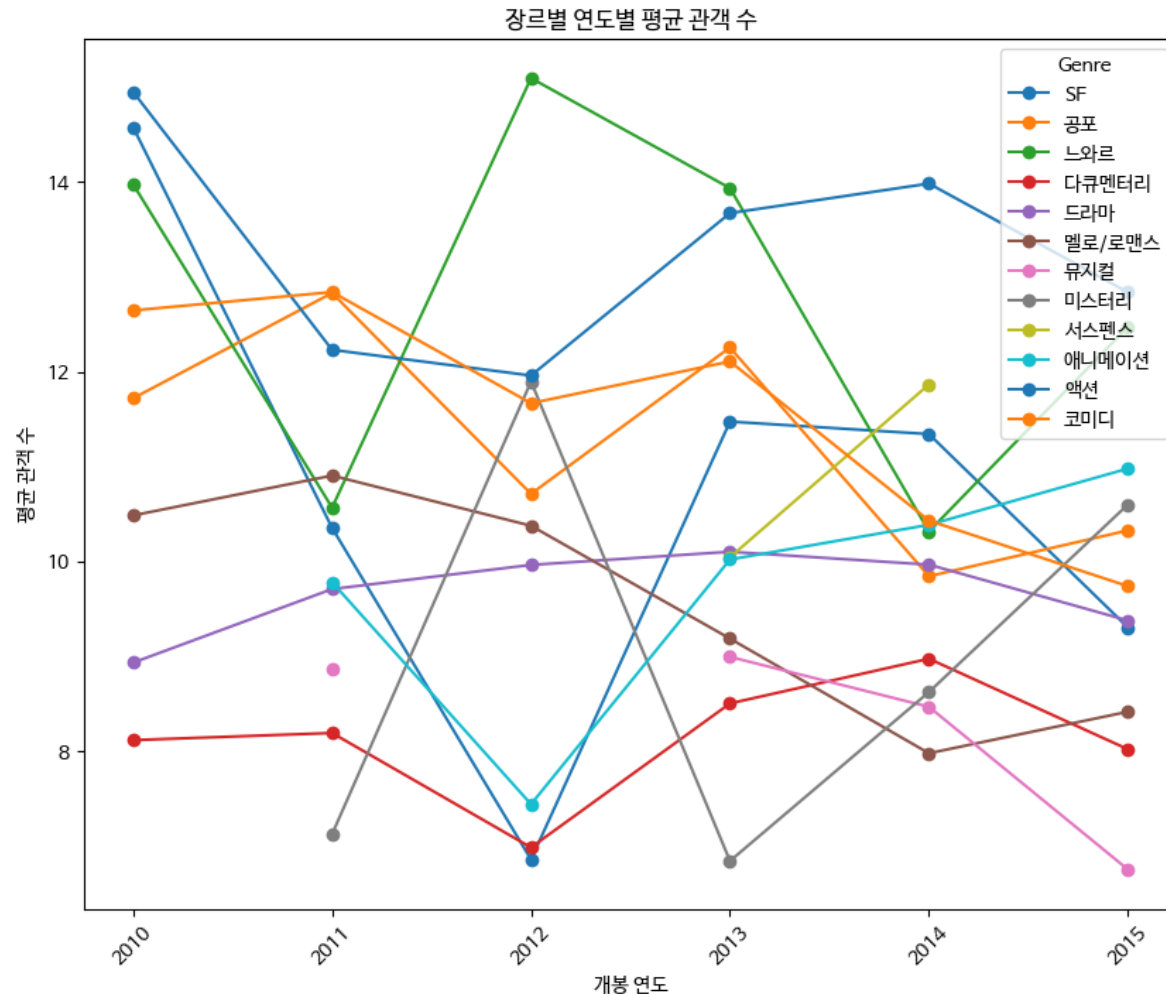
```
ax.set_ylabel("평균 관객 수")
```

```
plt.xticks(rotation=45)
```

```
plt.legend(title='Genre')
```

```
plt.show()
```

3-5. 장르별로 개봉 연도에 따른 관객 수



lineplot을 보면 SF와 느와르 장르는 변동이 큰 반면, 다큐멘터리와 드라마 장르는 안정적이다.

액션과 느와르 장르는 평균 관객 수가 높으나 다큐멘터리는 평균 관객 수가 낮다.

2012년에 느와르와 미스터리 장르의 평균 관객 수가 크게 증가한 것으로 보아 해당 장르의 흥행작이 있었을 것으로 예측할 수 있다.

3-5. 장르별로 개봉 연도에 따른 관객 수

```
genre_yearly = train.groupby(['genre', 'release_year'])['log_box_off_num'].mean().reset_index()
```

```
n = len(genre_yearly['genre'].unique())
```

```
cols = 2
```

```
rows = (n + 1) // cols
```

```
fig, axes = plt.subplots(rows, cols, figsize=(12,5*rows))
```

```
axes = axes.flatten()
```

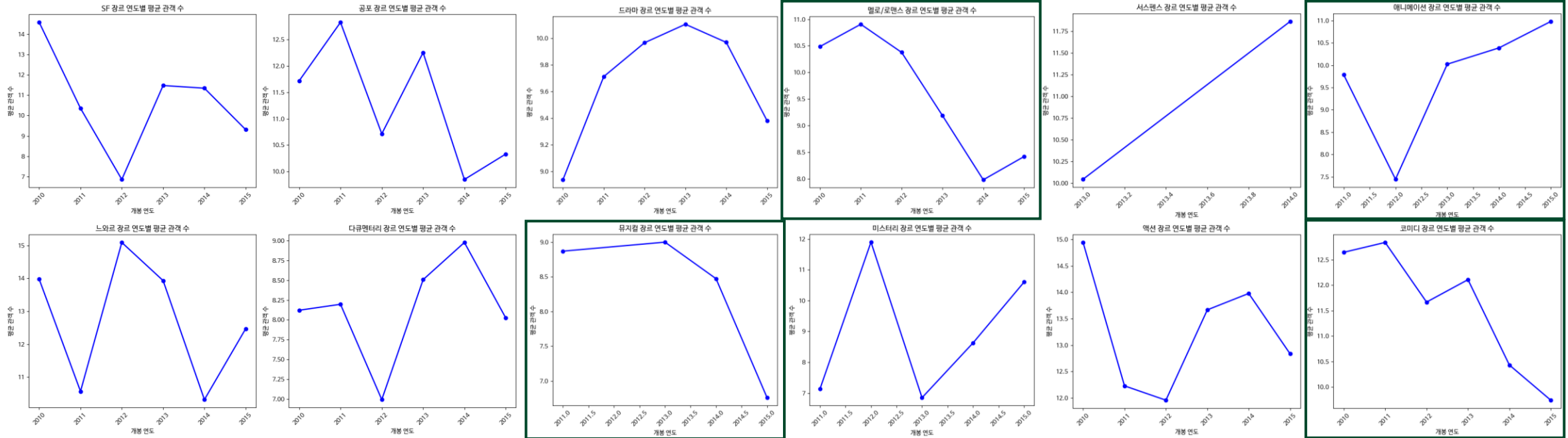
```
for i,g in enumerate(genre_yearly['genre'].unique()):
    data = genre_yearly[genre_yearly['genre'] == g]
    axes[i].plot(data['release_year'], data['log_box_off_num'], marker='o', color='blue', linewidth=2)
    axes[i].set_title(f"{g} 장르 연도별 평균 관객 수", fontsize=12)
    axes[i].set_xlabel("개봉 연도")
    axes[i].set_ylabel("평균 관객 수")
    axes[i].tick_params(axis='x', rotation=45)
```

```
plt.tight_layout()
```

```
plt.show()
```

enumerate 반복문을 이용해 장르별 데이터를 순회하며,
각 subplot에 장르별 연도별 평균 관객 수를 그려주도록 자동화

3-5. 장르별로 개봉 연도에 따른 관객 수



뮤지컬, 멜로/로맨스, 코미디 장르는 평균 관객 수가 감소하는 것을 볼 수 있는 반면 애니메이션 장르는 평균 관객 수가 증가하는 것을 확인할 수 있다.

3-6. 장르별 평균 관객 수

장르별 평균 관객 수와 분포를 알아보고자 바플롯과 박스플롯 그림

```
df['log_box_off_num'] = np.log1p(df['box_off_num'])
df_grouped=df.groupby('genre')['log_box_off_num'].mean()
df_grouped=df_grouped.sort_values(ascending=False)
fig, ax=plt.subplots(nrows=1,ncols=2,figsize=(15,8))
# barplot
sns.barplot(x=df_grouped.index, y=df_grouped.values, ax=ax[0])
ax[0].set_xlabel('장르')
ax[0].set_ylabel('관객수')
ax[0].set_title('장르별 평균 관객 수')
ax[0].set_xticklabels(df_grouped.index,rotation=45)
```

Box_off_num 로그처리

장르별 평균 관객 수

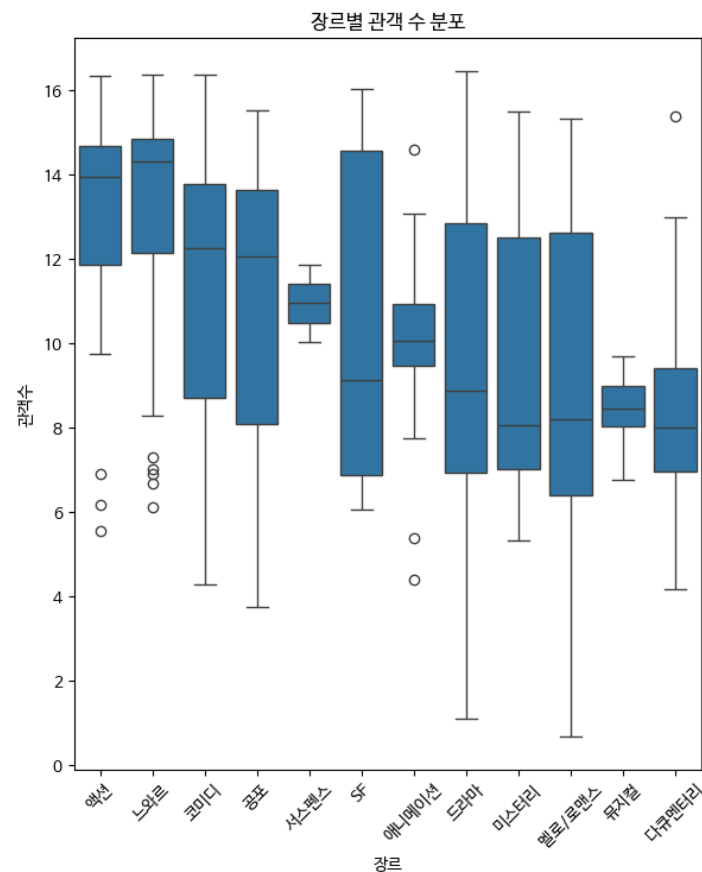
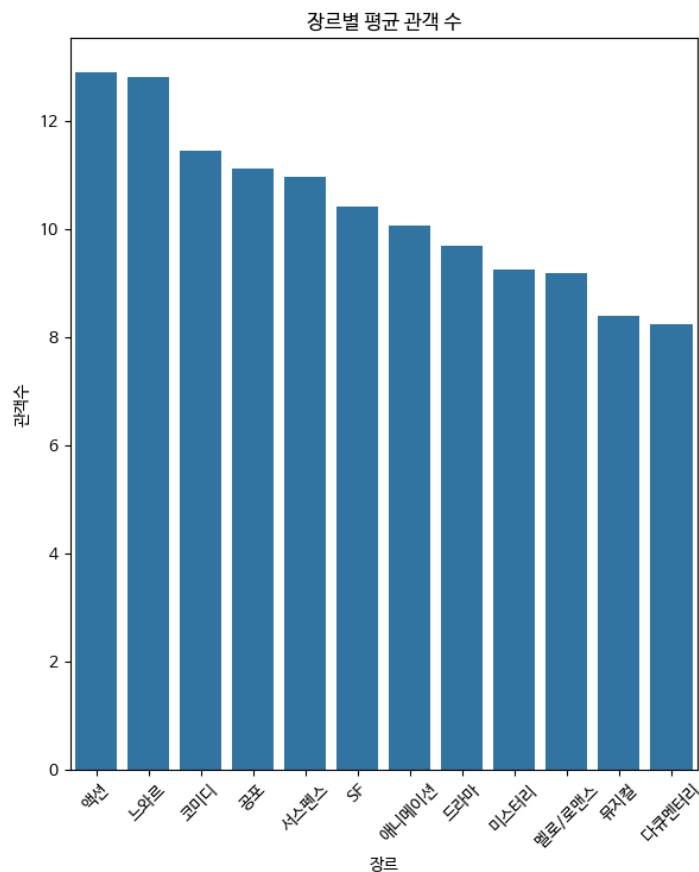
```
# boxplot
sns.boxplot(x='genre', y='log_box_off_num', data=df, order=df_grouped.index, ax=ax[1])
ax[1].set_xlabel('장르')
ax[1].set_ylabel('관객수')
ax[1].set_title('장르별 관객 수 분포')
ax[1].set_xticklabels(df_grouped.index,rotation=45)
```

장르별 관객 수 분포

```
plt.show()
```


3-6. 장르별 평균 관객 수

- 장르별 평균 관객 수: 액션>느와르>코미디>공포>서스펜스 순서로 많음
- 장르별 관객 수 중앙값과 평균은 약간의 차이를 보임
- 액션, 느와르: 평균치가 높고 분포가 위로 치우침> 흥행하는 경우가 많음
- 드라마, 멜로/로맨스: 분산이 큼> 흥행 불확실성이 높음
- 뮤지컬/다큐멘터리: 낮은 분포에 몰려 있으며 흥행을 기록하는 경우가 드물



3-7. 배급사별 평균 관객 수

Top10 배급사별 평균 관객 수와 분포를 알아보고자 바플롯과 박스플롯 그림

```
df_9 = df.groupby('distributor')['log_box_off_num'].mean()
df_9 = df_9.sort_values(ascending=False).iloc[:10]

fig, axes = plt.subplots(1, 2, figsize=(15,10))

# 왼쪽: 상위 10개 배급사 평균 관객 수 barplot
sns.barplot(x=df_9.index, y=df_9.values, ax=axes[0])
axes[0].set_title('배급사 별 평균 관객수')
axes[0].set_xlabel('배급사')
axes[0].set_ylabel('관객수')
axes[0].set_xticklabels(df_9.index, rotation=45)
axes[0].tick_params(axis='y', labelsize=10)

# 오른쪽: boxplot (분포 확인용)
sns.boxplot(x='distributor', y='log_box_off_num', data=df[df['distributor'].isin(df_9.index)], ax=axes[1])
axes[1].set_title('배급사 별 관객수 분포')
axes[1].set_xlabel('배급사')
axes[1].set_ylabel('관객수')
axes[1].set_xticklabels(df_9.index, rotation=45)

plt.tight_layout()
plt.show()
```

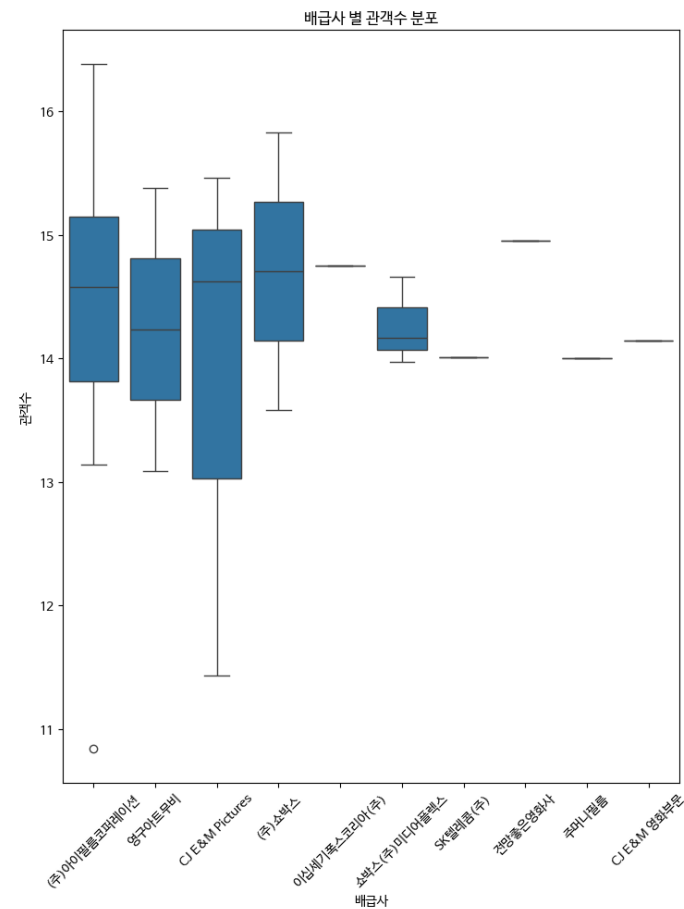
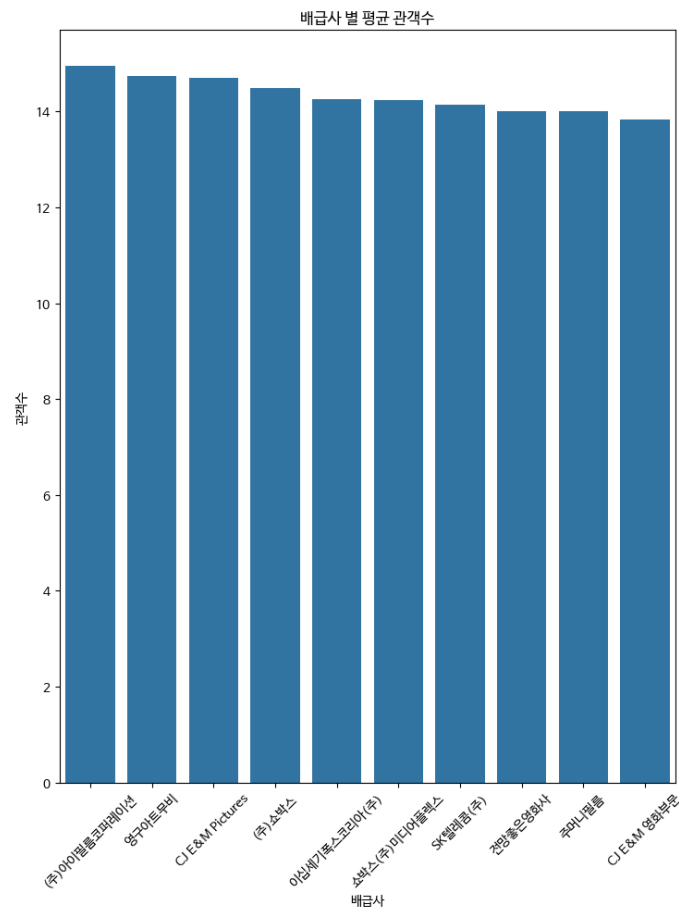
Log 관객 수 평균
Top10배급사 iloc

배급사별 평균 관객 수

배급사별 관객 수 분포

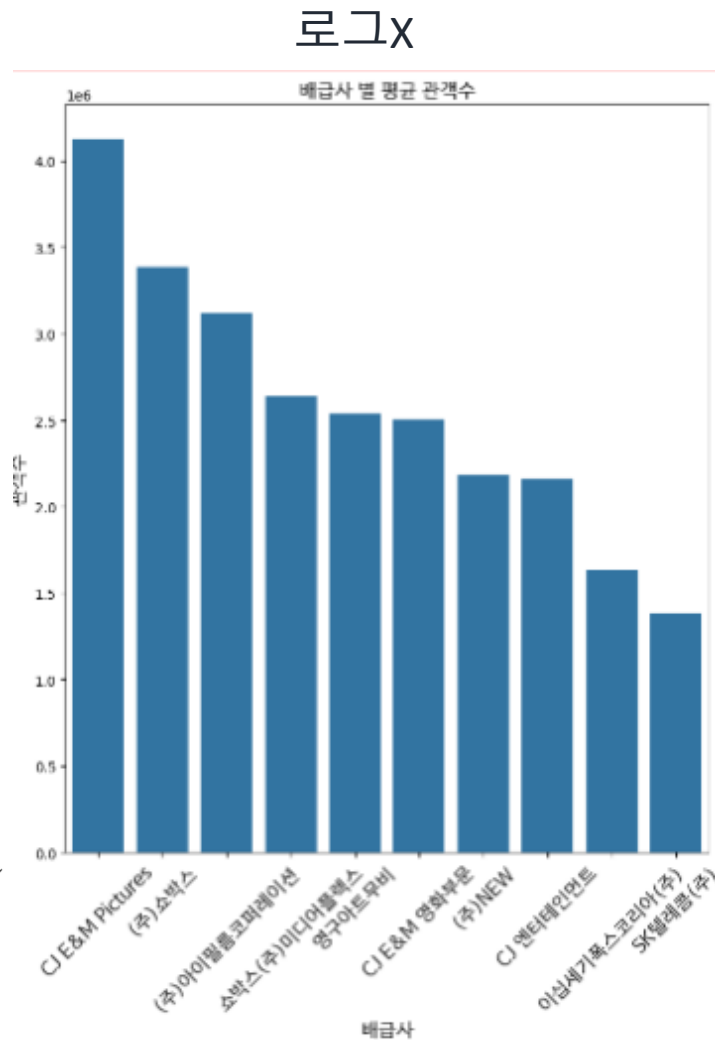
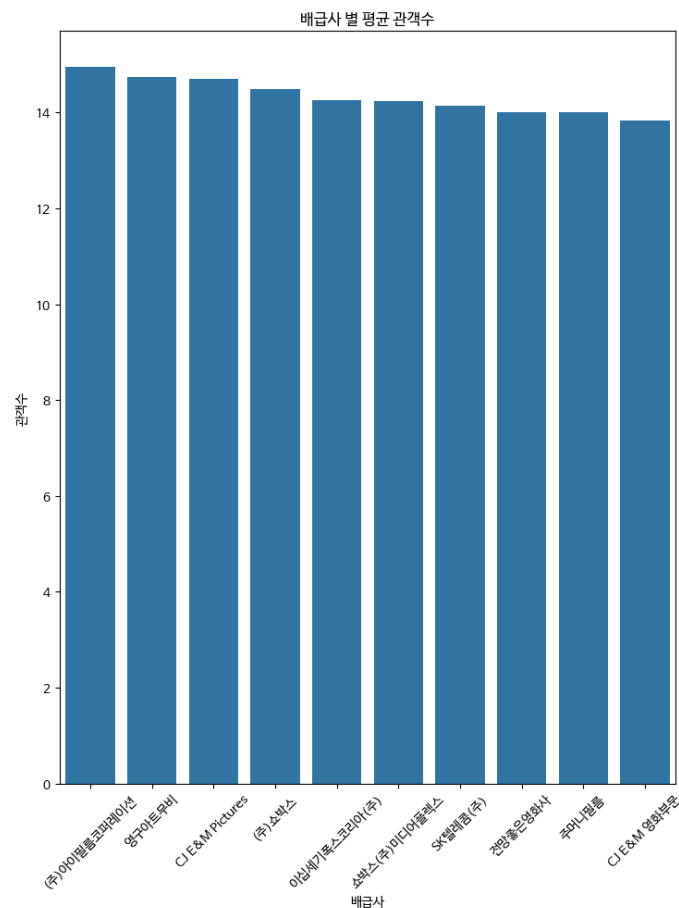
3-7. 배급사별 평균 관객 수

- (주)아이필름코퍼레이션, 영구아트무비, CJ E&M PICTURES, (주)쇼박스, 이십세기폭스코리아(주) 순서로 평균 관객 수가 많음
- CJ E&M PICTURES, (주)아이필름코퍼레이션은 관객 수 분포가 넓음> 영화 성과의 편차가 큼
- 중앙값과 평균값 순위 차이가 비교적 큼



3-7. 배급사별 평균 관객 수

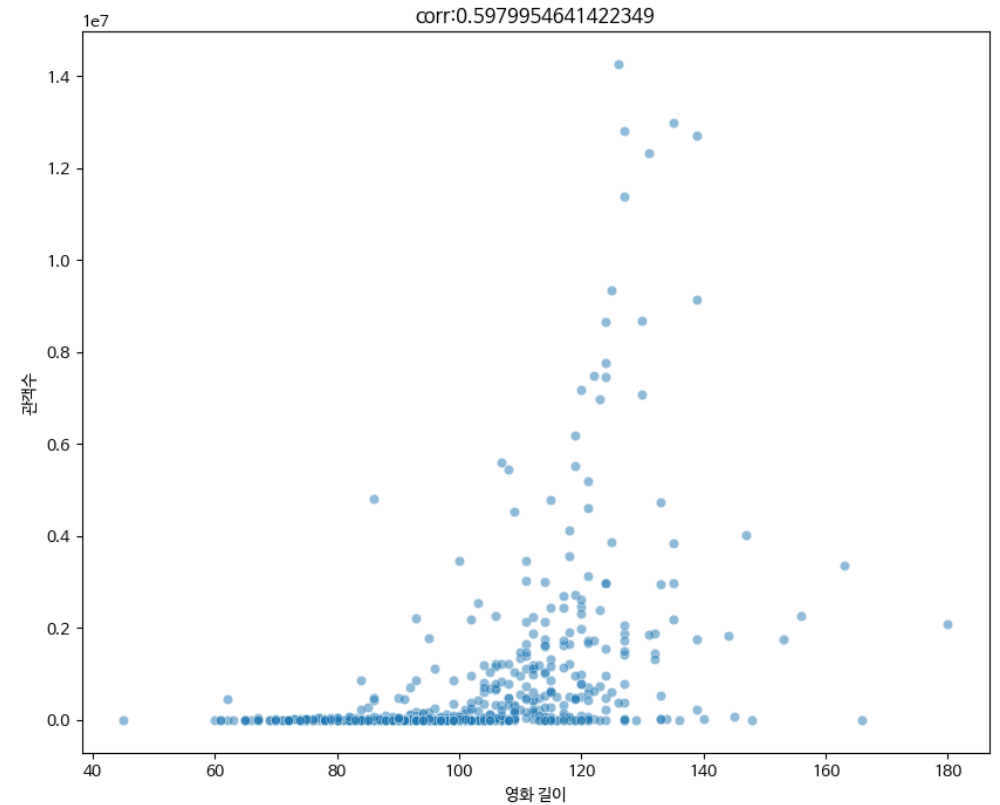
- 로그처리를 하지 않을 경우와 다른 배급사 별 평균 관객 수 순위를 보임
- 로그 처리O: (주)아이필름코퍼레이션, 영구아트무비, CJ E&M PICTURES, (주)쇼박스, 이십세기폭스코리아(주) 순
- 로그 처리X:CJ E&M PICTURES, (주)쇼박스, (주)아이필름코퍼레이션,쇼박스(주)미디어플렉스,영구아트무비 순



3-8. 영화 길이와 관객 수의 관계

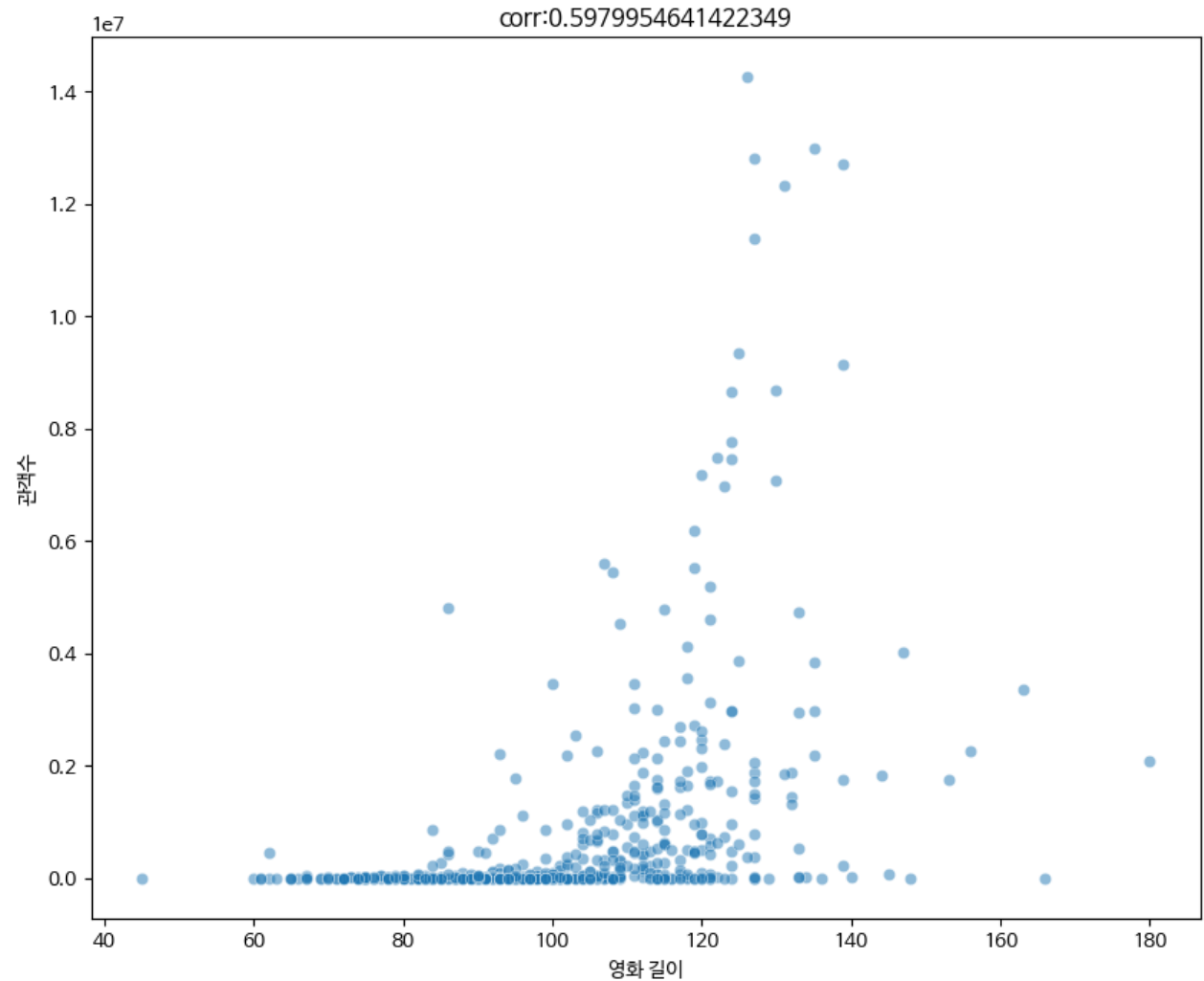
```
plt.figure(figsize=(10,8))
sns.scatterplot(x='time', y='box_off_num', data=df, alpha=0.5)
corr_num=df[['time', 'log_box_off_num']].corr().iloc[0,1]
plt.xlabel('영화 길이')
plt.ylabel('관객 수')
plt.title('corr: '+ str(corr_num))
plt.show()
```

영화 길이&관객 수 산점도
상관계수 계산



3-8. 영화 길이와 관객 수의 관계

- 상관계수 0.598: 영화 길이와 관객 수가 양의 상관관계를 보임
- 그러나 140분을 넘어가면 관객 수가 크게 적어짐
- 100-140분대 영화에 관객 수가 크게 몰려 있음
- 80분 이하의 영화에서는 거의 흥행 사례가 없음

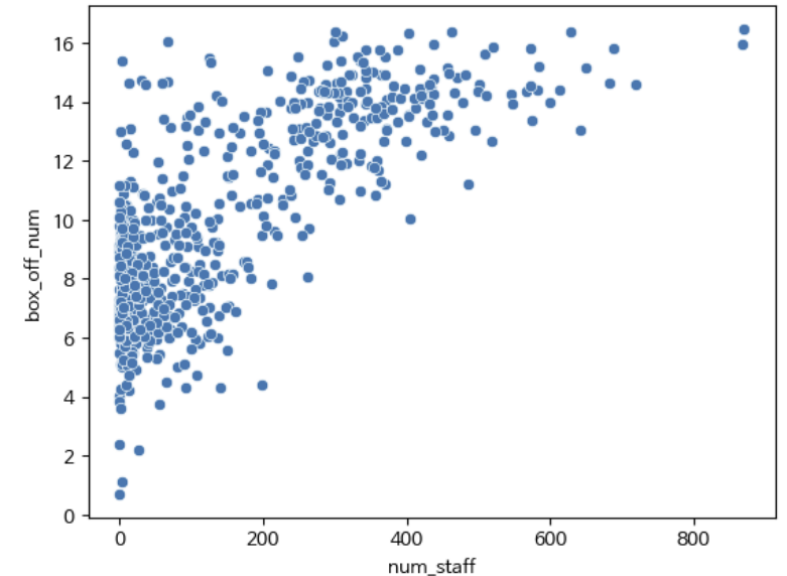


3-9. 스태프 수에 따른 관객 수

* 영화 제작 시 인력이 많이 투입될수록 더 많은 관객 수가 산출되는지 알아보기 위해 분석 실행

(a) 스캐터 플롯

```
train['box_off_num'] = np.log1p(train['box_off_num'])  
  
sns.scatterplot(x='num_staff', y='box_off_num', data=train)  
plt.show()
```



* 제작 인력 규모가 500명 이하인 영화들이 대다수

* 500명 이상의 인력이 투입된 영화 중에도 높은 관객 수를 기록한 작품들이 상당수 존재함

-> 인력 규모에 따라 흥행 성과에 어떤 차이가 있는지 심층적으로 분석하기 위해,

투입된 인력의 수를 일정한 구간으로 나누어 바이올린 플롯으로 시각화할 필요성 有

3-9. 스태프 수에 따른 관객 수

(b) 바이올린 플롯

```
train['num_staff'].max()
```

```
Out[ ]:  
869
```

```
# num_staff 구간 정의
```

```
bins = [0, 100, 200, 300, 400, 500, 900]
```

```
# 각 구간의 라벨 정의
```

```
labels = ['0-100', '100-200', '200-300', '300-400', '400-500', '500+']
```

num_staff 열을 구간으로 나누어 새로운 열을 생성하기 위해 num_staff의 최댓값을 파악함

```
train['staff_bin'] = pd.cut(train['num_staff'], bins=bins, labels=labels, right=False, include_lowest=True)
```


3-9. 스태프 수에 따른 관객 수

(b) 바이올린 플롯

```
# 계산된 평균을 막대 그래프로 시각화
plt.figure(figsize=(10, 6))
sns.violinplot(x='staff_bin', y='box_off_num', data=train, palette='viridis')

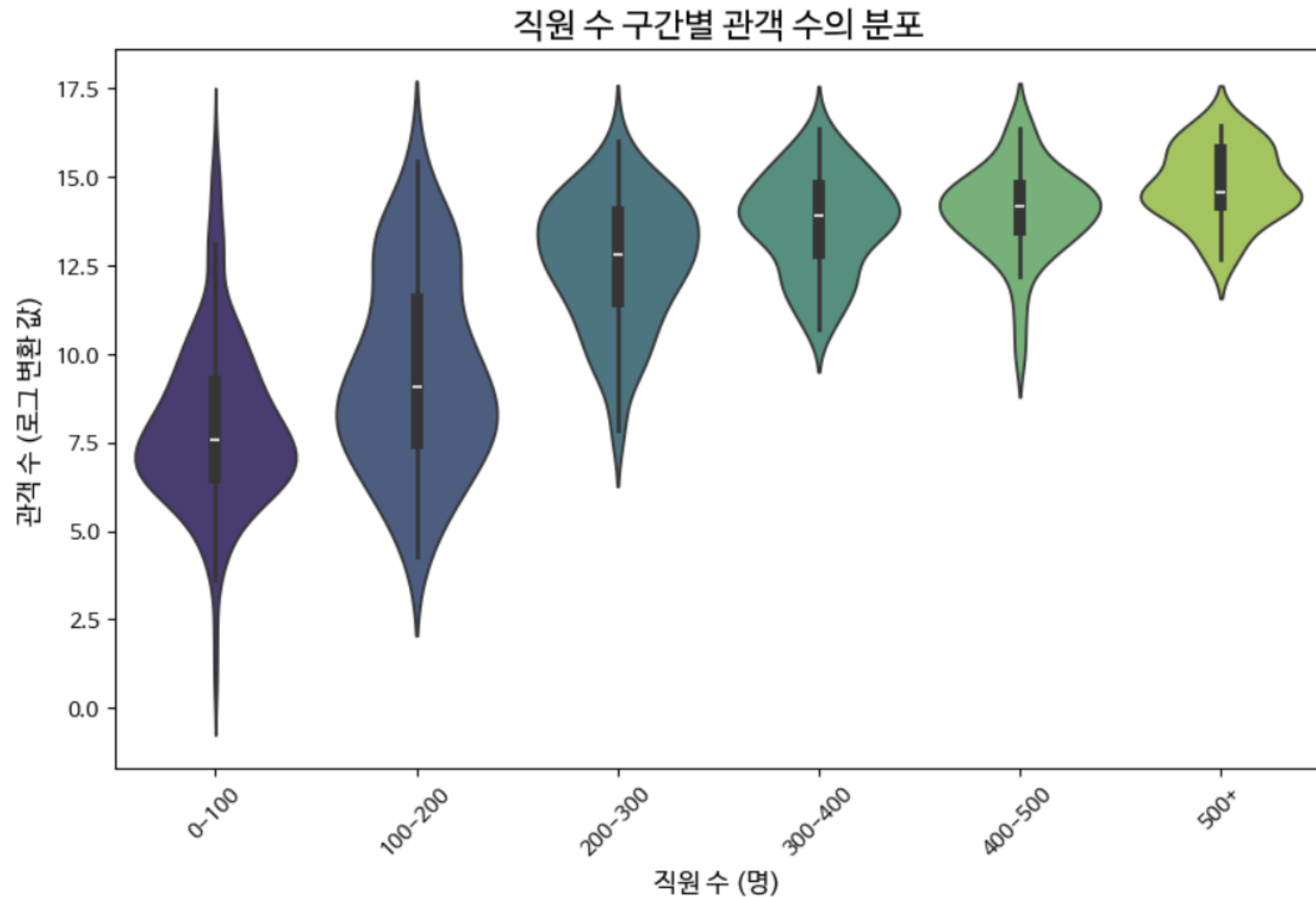
# 그래프 제목 및 라벨 설정
plt.title('직원 수 구간별 관객 수의 분포', fontsize=16)
plt.xlabel('직원 수 (명)', fontsize=12)
plt.ylabel('관객 수 (로그 변환 값)', fontsize=12)
plt.xticks(rotation=45) # x축 라벨이 겹치지 않도록 회전

# 그래프 표시
plt.show()
```

바이올린 플롯들의 색이
각각 다르게 나옴

3-9. 스태프 수에 따른 관객 수

(b) 바이올린 플롯



- 스태프 수가 증가할수록 바이올린 플롯의 높이가 증가하고, 폭도 넓어지는 경향이 있음
- 스태프 수가 많을수록 관객 수의 평균이 높아지지만, 편차도 커지기 때문에 흥행 성적의 양극화가 심해진다고 볼 수 있음

3-10. 배급사와 장르의 관계

- * 영화 제작 시 각 장르에 맞는 배급사를 선택하는 것은 흥행에 중요한 요소
- * 주요 배급사의 배급 영화 장르를 알아보기 위해 분석 실행

```
distributors = train['distributor'].unique()
n_distributors = len(distributors)
print(n_distributors)

top10_distributors = train['distributor'].value_counts().head(10).index
print(top10_distributors)
```

데이터프레임의 특정 열에서 중복되지 않는 유일한 값들만 추출

169

```
Index(['CJ 엔터테인먼트', '롯데엔터테인먼트', '(주)NEW', '(주)마운틴픽처스', '(주)쇼박스', '인디스토리',
      '골든타이드픽처스', '(주)케이알씨지', '시너지', '(주)키노아이'],
      dtype='object', name='distributor')
```

- 169개의 배급사가 있으나 그중 영화를 가장 많이 배급한 순서대로 top10 배급사를 골라 그들의 배급 현황을 파악함

3-10. 배급사와 장르의 관계

다차원 배열을 1차원 배열로 만들어줌
[[ax1, ax2], [ax3, ax4]] => [ax1, ax2, ax3, ax4]

```
axes = axes.flatten()
```

```
#enumerate(): top10_distributors 목록의 요소들을 인덱스와 함께 순회하게 해주는 파이썬 내장 함수  
for i, dist in enumerate(top10_distributors):
```

```
# 해당 배급사의 데이터만 필터링
```

```
dist_data = train[train['distributor'] == dist]
```

```
# 장르별 빈도수 계산
```

```
genre_counts = dist_data['genre'].value_counts()
```

```
# 막대 그래프 그리기 (ax=axes[i]: 그래프를 그릴 서브플롯의 위치를 지정함)
```

```
sns.barplot(x=genre_counts.index, y=genre_counts.values, ax=axes[i], palette='viridis')
```

```
# 그래프 제목 및 라벨 설정
```

```
axes[i].set_title('배급사: {}'.format(dist), fontsize=14)
```

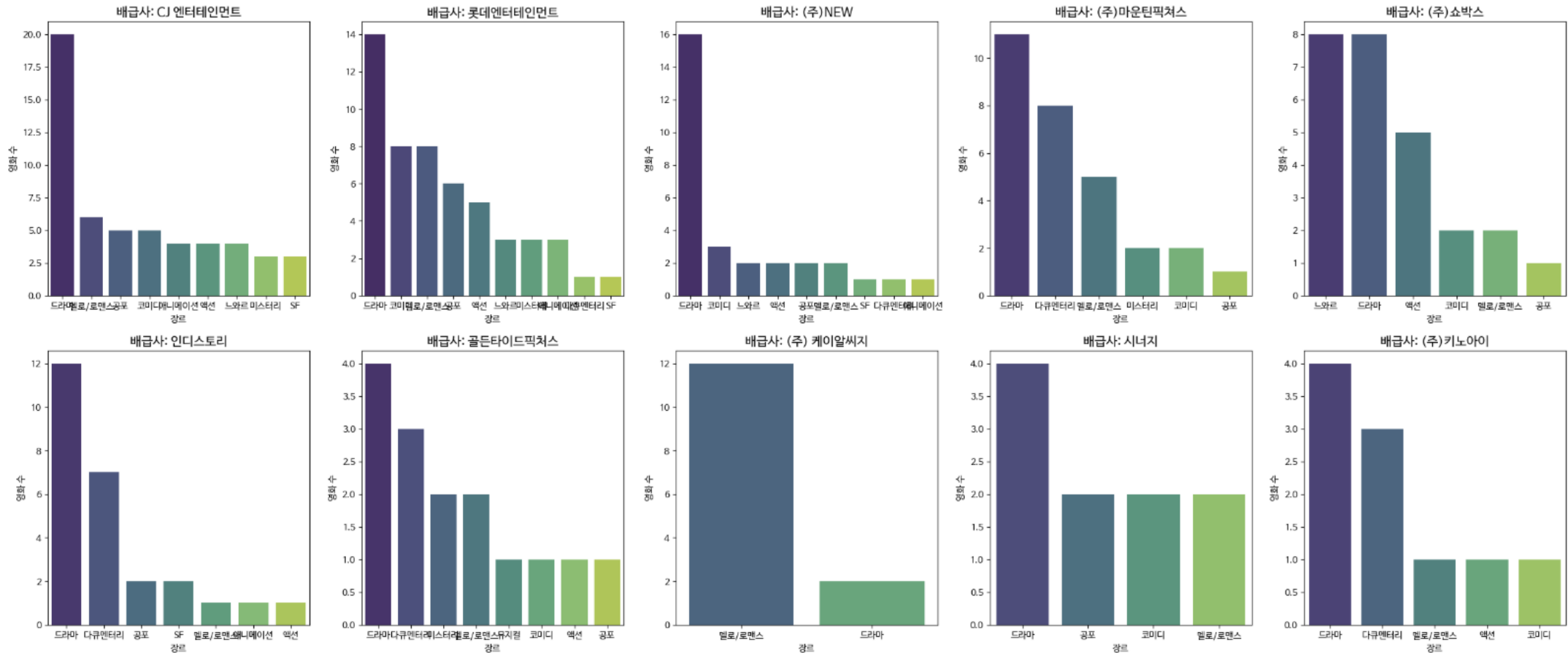
```
axes[i].set_xlabel('장르')
```

```
axes[i].set_ylabel('영화 수')
```

3-10. 배급사와 장르의 관계

서브플롯 & 바플롯

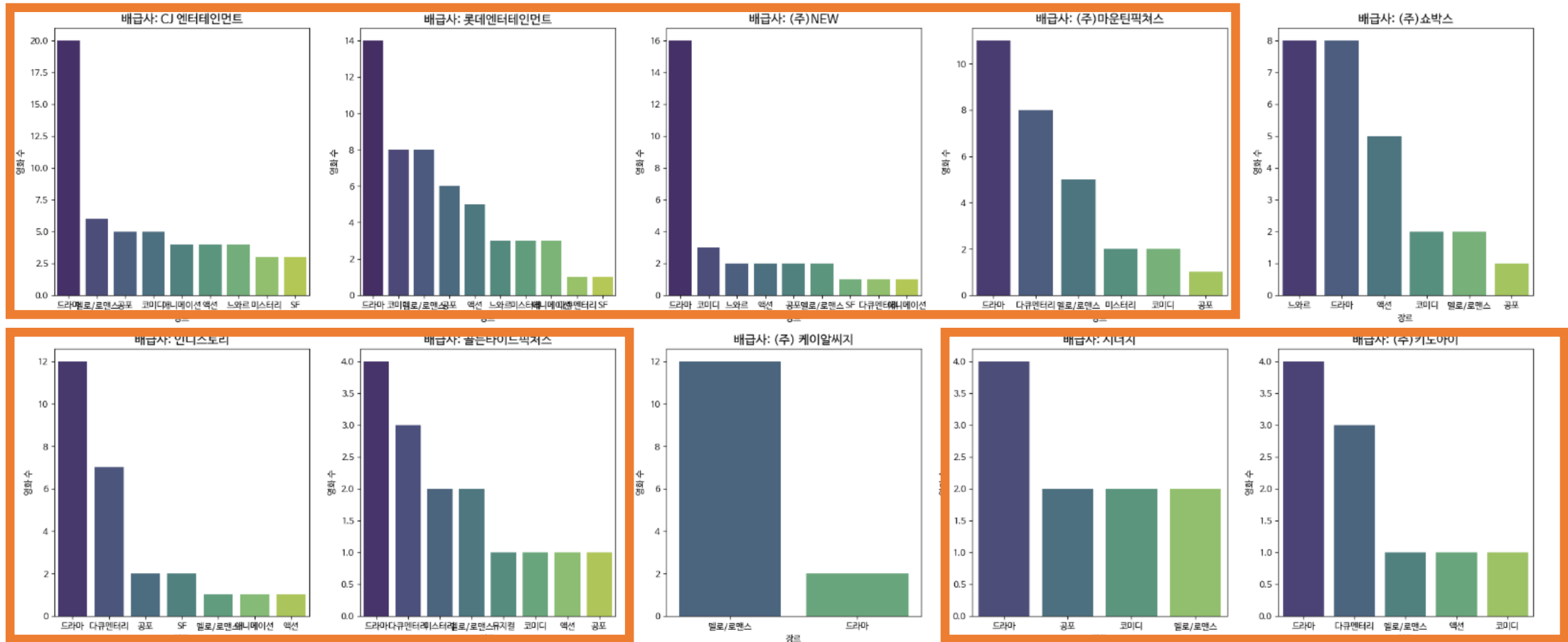
```
plt.show()
```



3-10. 배급사와 장르의 관계

서브플롯 & 바플롯

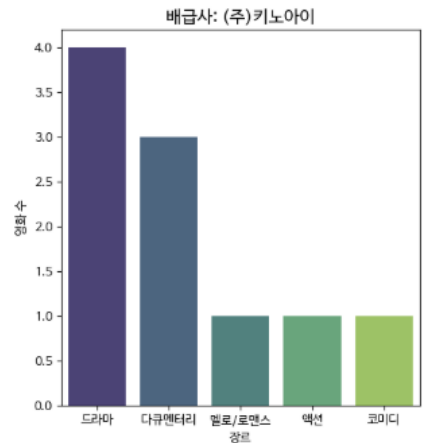
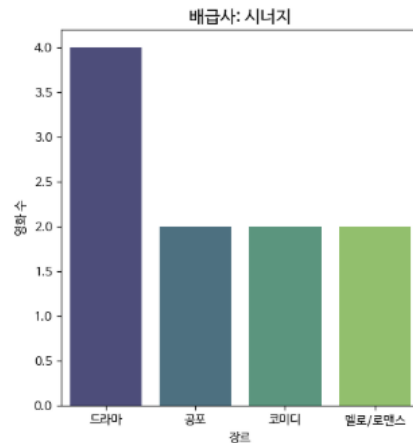
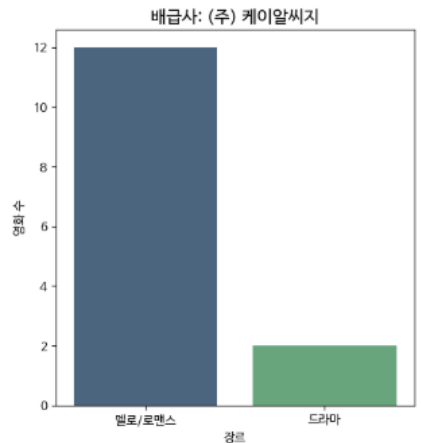
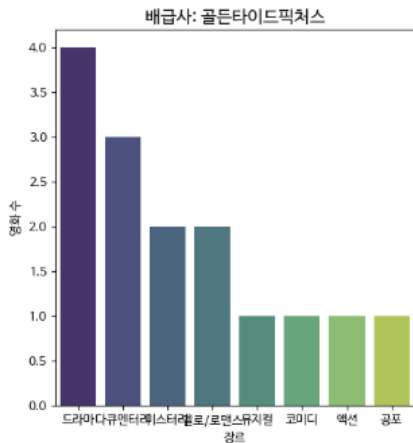
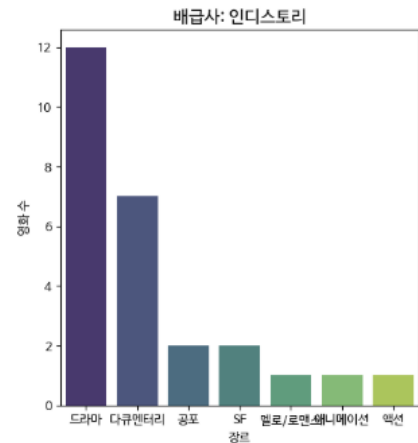
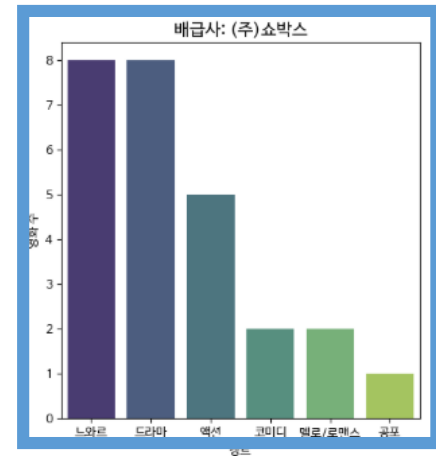
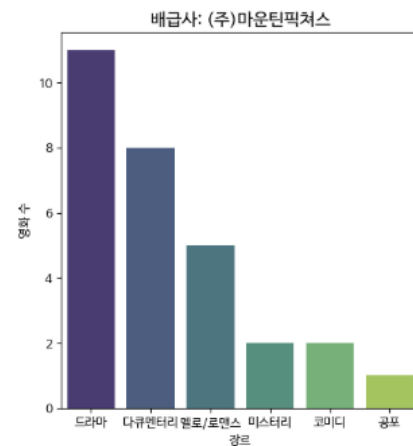
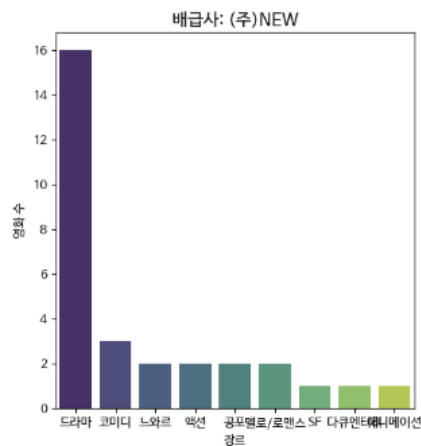
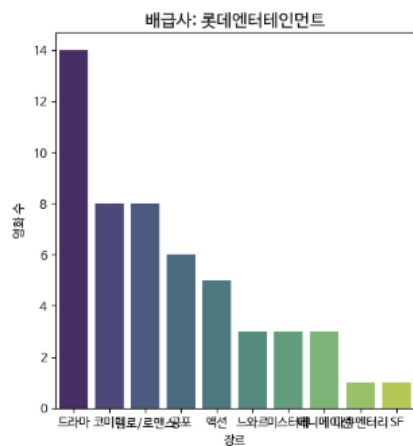
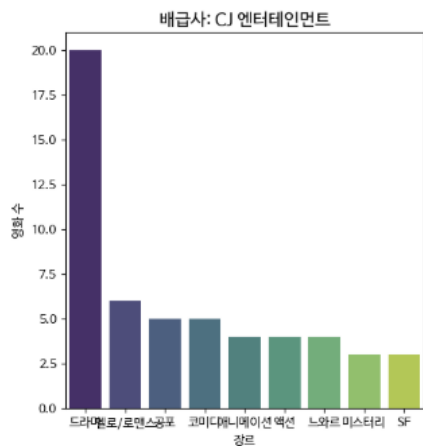
plt.show()



3-10. 배급사와 장르의 관계

서브플롯 & 바플롯

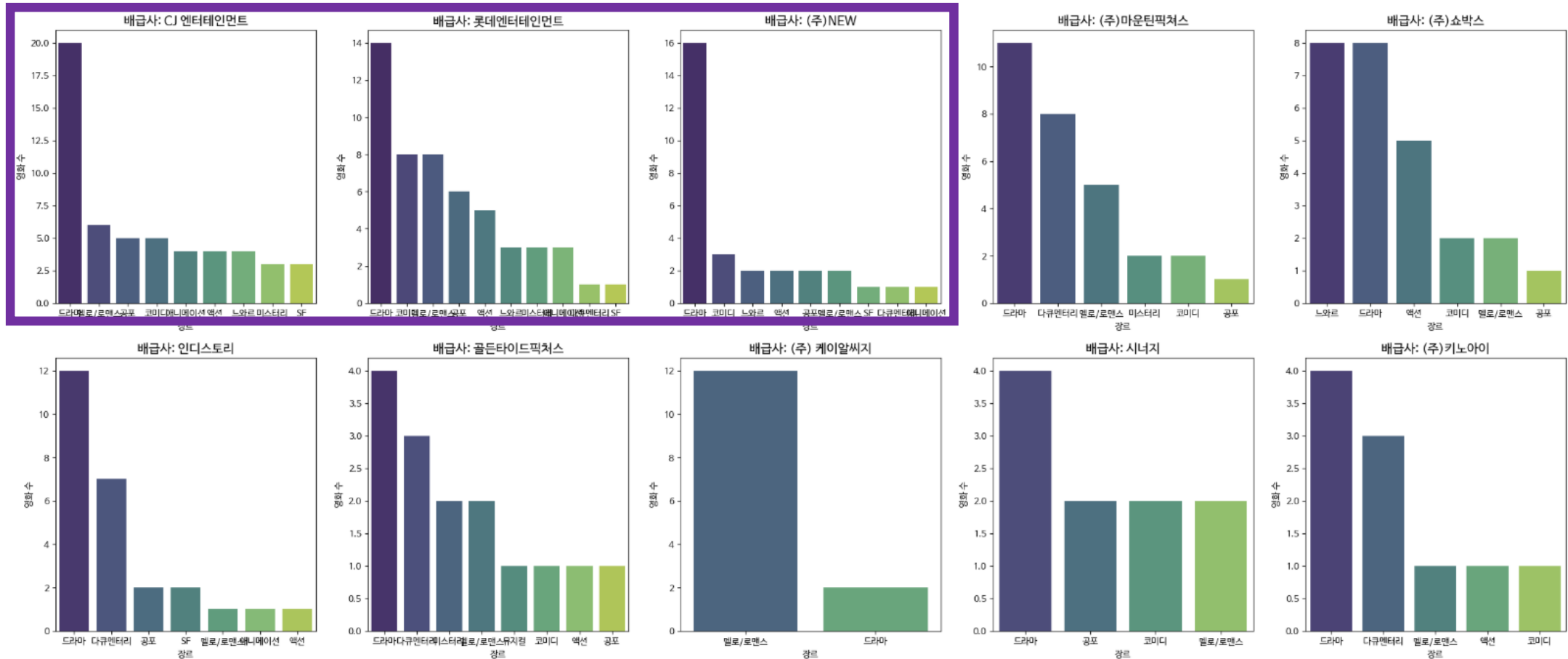
```
plt.show()
```



3-10. 배급사와 장르의 관계

서브플롯 & 바플롯

plt.show()





감사합니다